

PROJET ECOTRACK

ÉTUDE DE FAISABILITÉ

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822)

I. CONTEXTE DU PROJET ECOTRACK

✓ Critères RNCP Ce1.1.1 / A1.1 : Vision synthétique des objectifs et contraintes

La métropole, qui regroupe environ 500 000 habitants, assure la collecte des déchets ménagers via un parc de 2 000 conteneurs répartis sur 12 secteurs géographiques. La gestion actuelle repose sur des tournées planifiées à fréquence fixe, sans connaissance du niveau de remplissage réel des conteneurs. Cette approche génère deux dysfonctionnements majeurs : des collectes inutiles sur des conteneurs quasi vides (30 à 40 % des arrêts), et des débordements sur les points les plus sollicités, source de nuisances et de plaintes citoyennes. La plateforme s'adresse à quatre profils d'utilisateurs : 15 000 citoyens actifs, 50 agents de collecte, 10 gestionnaires et 3 administrateurs.

ECOTRACK répond à ce constat par une plateforme IoT de gestion intelligente des déchets. Chaque conteneur est équipé d'un capteur ultrasonique de niveau transmettant une mesure toutes les 15 minutes (ou sur changement significatif) via le protocole MQTT, soit environ 500 000 messages par jour. Ces données alimentent en temps réel une cartographie des niveaux de remplissage et un moteur d'optimisation des tournées fondé sur un algorithme du voyageur de commerce (TSP, raffiné par 2-opt), tenant compte de la capacité des véhicules, des horaires et des zones prioritaires.

La solution s'articule autour de six piliers (connectivité IoT, intelligence artificielle, mobilité, analytics, gamification et sécurité) et se décline en trois applications relevant de la filière Développement : un Dashboard web pour les gestionnaires et administrateurs (monitoring temps réel, création de tournées optimisées, analytics et rapports), une application mobile Citoyen (signalement de conteneurs pleins et gamification) et une application mobile Agent (réception de tournée, scan QR, géolocalisation et mode hors ligne). La gamification — points, badges, défis et leaderboard — constitue un module de développement central destiné à soutenir l'engagement des 15 000 citoyens.

L'objectif est de faire passer la collecte d'une logique calendaire à une logique pilotée par la donnée : ne collecter que les conteneurs qui le nécessitent, au meilleur moment et selon l'itinéraire le plus court. Les cibles visées sont -20 % de distances parcourues, un taux de débordements <2 %, -15 % de coûts opérationnels, -18 % d'émissions

CO₂, une disponibilité >99,5 % et 15 000 utilisateurs actifs. La présente étude évalue la faisabilité technique, économique et opérationnelle du projet afin de fonder une décision de lancement éclairée.

A. Problématique Actuelle

- Collecte inefficace : tournées fixes sans optimisation
- Débordements conteneurs : manque de visibilité temps réel
- Coûts élevés : carburant, main d'œuvre, maintenance
- Impact environnemental : émissions CO₂, pollution

B. Objectifs du Projet

- Réduire les coûts de collecte de 20-30%
- Éliminer les débordements (<5% incidents)
- Optimiser les tournées (algorithme TSP)
- Fournir des analytics temps réel aux gestionnaires

II. FAISABILITÉ TECHNIQUE

✓ Critères RNCP Ce1.1.2 / A1.3 : Analyse contraintes techniques et faisabilité

Faisabilité Technique Applicative (Filière Développement)

A. Stack Backend

- Framework : Node.js (services temps réel : ingestion IoT, notifications) + FastAPI (Python) pour l'API métier et l'optimisation des tournées
- Base de données : PostgreSQL 15 + extension PostGIS (géospatial : conteneurs, zones, tournées)
- Cache : Redis pour sessions, classements de gamification et requêtes fréquentes (objectif cache hit >80 %)
- Ingestion IoT : capteurs ultrasoniques sur 2 000 conteneurs émettant via MQTT toutes les 15 min (ou sur changement significatif), soit ~500 000 messages/jour, relayés sur un bus Kafka avant persistance

B. Stack Frontend

- Framework : React 18 + TypeScript + TailwindCSS (Dashboard web) ; React Native pour les applications mobiles Citoyen et Agent
- Cartographie : Leaflet + OpenStreetMap (états des conteneurs, itinéraires, heatmaps)
- Temps réel : WebSocket (Socket.io) pour push notifications
- Charts : Recharts pour le dashboard analytics (KPIs temps réel, évolutions)

C. Algorithme Optimisation Tournées

- TSP (Traveling Salesman Problem) : heuristique gloutonne + raffinement 2-opt pour le MVP, algorithme génétique en évolution
- Contraintes : capacité véhicule, horaires, zones prioritaires
- Performance : calcul tournée optimale <30s pour 50 conteneurs

D. Scalabilité et Performance

- Architecture microservices (Domain-Driven Design) derrière une API Gateway : services IoT, Optimisation, API métier, Auth, Notification et Gamification, communiquant via bus Kafka
- Containerisation : Docker + Docker Compose
- Load balancing : Nginx reverse proxy

E. Sécurité et Authentification

- API REST documentée (OpenAPI/Swagger), authentification JWT et contrôle d'accès RBAC à 4 rôles (citoyen, agent, gestionnaire, administrateur)

- Mots de passe chiffrés bcrypt (10 rounds), rate limiting 100 req/min/IP, conformité OWASP Top 10, RGPD, WCAG 2.1 AA et ISO 27001

F. Gamification et Applications Mobiles

- Module de gamification : attribution de points, badges, défis collectifs et leaderboard, exposé via une API dédiée et appuyé sur Redis pour les classements
- Applications mobiles React Native : App Citoyen (signalement + gamification) et App Agent (tournée, scan QR, géolocalisation, mode hors ligne avec synchronisation différée)

G. Qualité et Industrialisation

- Tests automatisés (unitaires, intégration, e2e) avec couverture >60 % et pipeline CI/CD GitHub Actions (build, test, déploiement conteneurisé Docker)
- Performance cible : API <500 ms (p95), requêtes SQL <200 ms (90 %), chargement page <3 s, 500 utilisateurs simultanés, uptime 99,5 % (RTO <4 h, RPO <1 h)

III. FAISABILITÉ ÉCONOMIQUE

✓ Critères RNCP Ce1.1.3 / A1.3 : Analyse faisabilité économique et ROI

A. Coûts de Développement (Année 1)

1. Ressources Humaines (16 semaines = 4 mois)

- Product Owner (25 % temps) : 7 k€
- Scrum Master (25 % temps) : 7 k€
- Développeurs full-stack (3 × 100 %) : 72 k€
- Architecte logiciel (50 % temps) : 16 k€
- DevOps/SRE (50 % temps) : 14 k€

SOUS-TOTAL RH : 116 k€

2. Infrastructure Cloud (4 mois développement)

- Serveurs (3 VM t3.medium AWS/Azure) : 1,5 k€
- Base de données (RDS PostgreSQL) : 1 k€
- Stockage (S3/Blob 100 Go) : 0,2 k€
- Bande passante : 0,3 k€

SOUS-TOTAL Infrastructure Dev : 3 k€

3. Licences et Outils

- Jira (5 users × 4 mois) : 0,2 k€
- GitHub Team (5 users) : 0,1 k€
- Monitoring (Datadog) : 1 k€

- Licences diverses (IDE, design, etc.) : 0,7 k€

SOUS-TOTAL Licences : 2 k€

TOTAL ANNÉE 1 (Développement) : 121 k€

B. Coûts Récurrents (Années 2-3)

1. Infrastructure Production (par an)

- Hébergement cloud (production + staging) : 12 k€/an
- Base de données : 4 k€/an
- CDN (CloudFlare) : 1 k€/an
- Backups et stockage : 1 k€/an

SOUS-TOTAL Infrastructure Prod : 18 k€/an

2. Maintenance et Support (par an)

- Équipe support (2 personnes 50 %) : 72 k€/an
- Évolutions fonctionnelles : 15 k€/an
- Corrections bugs : 5 k€/an
- Mises à jour sécurité : 5 k€/an

SOUS-TOTAL Maintenance : 97 k€/an

3. Licences Récurrentes (par an)

- Outils monitoring/alerting : 3 k€/an
- Licences SaaS : 2 k€/an

SOUS-TOTAL Licences : 5 k€/an

TOTAL PAR AN (Années 2-3) : 120 k€/an

C. TCO (Total Cost of Ownership) 3 ans

- Année 1 (Développement) : 121 k€
 - Année 2 (Production) : 120 k€
 - Année 3 (Production) : 120 k€
-

—
TCO 3 ANS : 361 k€

D. Économies Estimées

Économies annuelles attendues :

- Optimisation des tournées : -20 % de distances parcourues (carburant + usure véhicules) = 90 k€/an
 - Réduction des débordements à <2 % : moins d'interventions d'urgence = 50 k€/an
 - Gains de productivité : -15 % de coûts opérationnels (main-d'œuvre mieux pilotée) = 60 k€/an
 - Bénéfices environnementaux et image : -18 % d'émissions CO₂, baisse des plaintes citoyennes = 20 k€/an
-

—
TOTAL ÉCONOMIES : 220 k€/an

E. Calcul du ROI

Formule : $ROI = (Gains\ cumulés - Investissement) / Investissement \times 100$

- Investissement initial (Année 1) : 121 k€
- Économies cumulées sur 3 ans : $220\text{ k€/an} \times 3 = 660\text{ k€}$
- Coûts récurrents 2 ans : $120\text{ k€/an} \times 2 = 240\text{ k€}$
- Gain net 3 ans : $660\text{ k€} - 361\text{ k€} = 299\text{ k€}$

ROI 3 ANS : 83 %

Seuil de rentabilité atteint : 2,1 ans

IV. FAISABILITÉ OPÉRATIONNELLE

A. Organisation de l'Équipe Projet

1. Composition de l'Équipe

- Product Owner : 1 personne (profil métier gestion déchets)
 - Scrum Master : 1 personne (certification PSM I)
 - Développeurs Full-Stack : 3 personnes (React + Node.js/Python)
 - Architecte Logiciel : 1 personne (expérience microservices)
 - DevOps/SRE : 1 personne (Docker, Kubernetes, CI/CD)
- TOTAL : 7 personnes (5.5 ETP)

2. Répartition des Rôles et Responsabilités

- PO : Backlog, user stories, validation métier
- SM : Animation Scrum, levée blocages, amélioration continue
- Devs : Développement features, tests, code reviews
- Archi : Conception architecture, choix techniques, POCs
- DevOps : CI/CD, infra, monitoring, déploiements

B. Méthodologie de Gestion de Projet

- Framework : Scrum (sprints 2 semaines)
- Durée projet : 16 semaines sur 4 mois (Kick-off S1, Sprint 0 S2-3, puis sprints de 2 semaines)
- Rituels : Daily (15 min), Planning (2h), Review (1h), Rétro (1h)
- Outils : Jira + Confluence + Slack + GitHub
- Vitesse cible : 25 ± 3 Story Points par sprint

C. Planning Prévisionnel

Phase 1 (S1-S7) : Cadrage, socle technique et modules core

- Kick-off (S1) + Sprint 0 (S2-3) : équipes, backlog, architecture microservices, environnement et CI/CD
- Sprint 1 (S4-5) : ingestion MQTT/Kafka, modèle de données PostGIS, API REST + Auth JWT/RBAC
- Sprint 2 (S6-7) : dashboard carte temps réel, alertes, tests de charge

Phase 2 (S8-S16) : optimisation, gamification, mobile et finalisation

- Sprints 3-4 (S8-11) : optimisation TSP+2-opt, dashboard analytics, module gamification, apps mobiles Citoyen et Agent
- Sprint 5 (S12-14) : intégration, tests e2e (coverage >60 %), déploiement, polish
- Soutenance (S15-16) : présentation finale et livraison du DCT (30-35 pages)

D. Risques Opérationnels

1. Départ membre clé (Prob: Faible, Impact: Élevé)
Mitigation : Documentation continue, pair programming, knowledge sharing
2. Retard développement fonctionnalité critique (Prob: Moyenne, Impact: Élevé)
Mitigation : MVP simplifié, priorisation MoSCoW, buffer 10% sur planning
3. Résistance utilisateurs finaux (Prob: Moyenne, Impact: Moyen)
Mitigation : Formation, beta testing avec chauffeurs, feedback loops
4. Intégration SI existant complexe (Prob: Élevée, Impact: Moyen)
Mitigation : APIs bien documentées, adapters, tests intégration
5. Dépassement budget (Prob: Moyenne, Impact: Élevé)
Mitigation : Suivi hebdomadaire budget, alertes à 80%, réserve 10%
6. Indisponibilité de la plateforme cloud (Prob : Faible, Impact : Élevé)
Mitigation : architecture multi-instances containerisée, supervision proactive (alertes), procédure de reprise documentée et sauvegardes régulières.
7. Non-conformité RGPD des données de géolocalisation (Prob : Moyenne, Impact : Élevé)
Mitigation : privacy by design, minimisation et chiffrement des données personnelles, registre de traitement et audit de conformité avant mise en production.

E. Facteurs Clés de Succès

- ✓ Engagement fort du sponsor projet (Directeur Gestion Déchets)
- ✓ Équipe expérimentée et motivée
- ✓ Méthodologie Agile bien maîtrisée
- ✓ Implication utilisateurs finaux dès Sprint 2
- ✓ Architecture scalable et évolutive
- ✓ Tests et validation continue
- ✓ Communication transparente avec parties prenantes

V. CONCLUSION ET RECOMMANDATIONS

A. Synthèse de la Faisabilité

DÉCISION : GO

Faisabilité Technique : FAVORABLE

Stack mature et éprouvée (React 18/React Native, Node.js + FastAPI, PostgreSQL 15 + PostGIS, Redis, Kafka), entièrement open source et déployée en architecture microservices conteneurisée (Docker). Les briques les plus complexes (ingestion MQTT à ~500 000 messages/jour via Kafka, optimisation TSP+2-opt, gamification) sont des problèmes documentés disposant de bibliothèques de référence. Les NFR (API <500 ms p95, 500 users simultanés) sont atteignables, les risques techniques étant atténués par une approche MVP et des tests de charge dès le sprint 3.

Faisabilité Économique : FAVORABLE

ROI estimé à 83 % sur 3 ans avec un seuil de rentabilité atteint en 2,1 ans. Les économies annuelles (220 k€/an : -20 % de distances, débordements ramenés à <2 %, -15 % de coûts opérationnels, -18 % de CO₂) dépassent largement les coûts récurrents (120 k€/an), pour un investissement initial maîtrisé de 121 k€.

Faisabilité Opérationnelle : FAVORABLE

Équipe pluridisciplinaire de 7 personnes (5,5 ETP) maîtrisant Scrum et la stack retenue. La méthodologie Agile, l'implication des agents de collecte dès les sprints 3-4 et un plan de gestion des risques structuré sécurisent l'exécution sur les 16 semaines.

Conclusion Globale :

L'analyse converge vers une décision de lancement (GO). Sur le plan technique, la solution s'appuie sur des technologies matures et open source organisées en microservices (API Gateway, bus Kafka, PostgreSQL+PostGIS, Redis, Docker), dont la combinaison répond précisément aux contraintes du projet (géospatial, ingestion IoT à ~500 000 messages/jour, temps réel, gamification, applications mobiles). Les exigences de performance (API <500 ms p95, 500 utilisateurs simultanés, uptime 99,5 %) sont réalistes au regard de cette architecture.

Sur le plan économique, le projet est rentable : un investissement initial de 121 k€ génère un ROI de 83 % sur trois ans, avec un retour sur investissement atteint dès la deuxième année grâce à des économies annuelles estimées à 220 k€ (-20 % de distances, débordements <2 %, -15 % de coûts opérationnels, -18 % de CO₂).

Sur le plan opérationnel, l'équipe et la méthodologie Scrum (sprints de 2 semaines sur 4 mois) sont adaptées au projet. Les principaux points de vigilance — montée en charge de l'ingestion IoT, adoption par les agents de collecte et conformité RGPD — sont identifiés et accompagnés de mesures d'atténuation concrètes.

En conséquence, le projet ECOTRACK est recommandé au lancement, sous réserve du respect d'une approche MVP, d'un suivi budgétaire hebdomadaire et de l'intégration de la sécurité et de l'éco-conception dès la phase de conception.

B. Recommandations Stratégiques

1. Approche MVP (Minimum Viable Product)

- Lancer avec fonctionnalités essentielles (cartographie + alertes + tournées simplifiées)
- Itérer rapidement basé sur feedback utilisateurs réels
- Éviter le perfectionnisme qui retarde la mise en production

2. Implication Continue des Utilisateurs Finaux

- Beta testing dès Sprint 4 avec 10 chauffeurs pilotes
- Feedback loops hebdomadaires
- Formation progressive avant déploiement général

3. Architecture Scalable dès le Début

- Microservices + containerisation pour faciliter évolutions futures
- Load testing dès Sprint 3 pour anticiper problèmes performance
- Design for failure (résilience, monitoring, alertes)

4. Sécurité et Conformité RGPD Natives

- Privacy by design dès conception
- Chiffrement données personnelles (chauffeurs, citoyens)
- Audit sécurité externe avant mise en production

5. Documentation Technique Continue

- Architecture Decision Records (ADR) pour choix techniques
- README.md à jour + documentation API (Swagger/OpenAPI)
- Runbooks pour incidents courants

6. Tests Automatisés Dès Sprint 1

- Couverture tests unitaires $\geq 80\%$
- Tests intégration + E2E pour parcours critiques
- CI/CD avec validation automatique avant merge

7. Plan de Communication Proactif

- Rapport hebdomadaire aux sponsors (KPIs, risques, décisions)
- Démonstrations sprint toutes les 2 semaines
- Transparence totale sur difficultés rencontrées

FIN DE L'ÉTUDE DE FAISABILITÉ

PROJET ECOTRACK

VEILLE TECHNOLOGIQUE

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822)

I. MÉTHODOLOGIE DE VEILLE TECHNOLOGIQUE

✓ Critères RNCP Ce1.2.1 / A1.2 : Veille structurée et documentée

La veille technologique d'ECOTRACK est conduite de manière structurée et continue afin de fonder chaque choix de stack sur des données objectives et actualisées. Elle repose en priorité sur des sources anglophones de référence, qui concentrent l'essentiel de l'information primaire et la plus récente : documentations officielles des projets (React, FastAPI, PostgreSQL/PostGIS, TimescaleDB, Redis, Mosquitto), benchmarks indépendants (TechEmpower Web Framework Benchmarks), rapports annuels (Stack Overflow Developer Survey, State of JS, ThoughtWorks Technology Radar) et dépôts GitHub des projets évalués (étoiles, fréquence des commits, issues ouvertes, cadence des releases). La maîtrise de l'anglais technique est ici une compétence transversale décisive, l'information francophone étant souvent partielle ou décalée dans le temps.

Les critères de sélection des technologies sont définis a priori et appliqués uniformément : maturité et pérennité, performance mesurée, adéquation aux contraintes du projet (géospatial, séries temporelles, temps réel IoT), taille et vitalité de la communauté, qualité de la documentation, coût total et licence (open source privilégié), et facilité d'intégration dans une architecture microservices conteneurisée. La veille s'organise selon trois rythmes : un scan quotidien de 15 à 30 minutes (agrégateurs RSS, newsletters, GitHub Trending), une analyse hebdomadaire approfondie de 2 heures (lecture de benchmarks et articles de fond), et une synthèse mensuelle partagée en équipe lors d'une Tech Watch. Les outils utilisés sont Feedly (agrégation de plus de 50 sources), Google Alerts sur les mots-clés stratégiques, les comptes d'experts sur X/Twitter et Reddit (r/programming, r/PostgreSQL), et Notion pour la capitalisation structurée des découvertes.

A. Sources Consultées

Sources Techniques :

- GitHub Trending : repos populaires et émergents
- Stack Overflow : questions/réponses trending
- Medium / Dev.to (anglophone) : retours d'expérience et articles techniques récents

- Documentation officielle (anglophone) : React, FastAPI, PostgreSQL, TimescaleDB, MQTT

Conférences et Événements :

- Conférences tech (anglophones) : React Summit, PyCon, FOSDEM, PGConf
- Webinars : Cloud providers (AWS, Azure, GCP)
- Meetups locaux : Paris JS, Python Paris, PostgreSQL France

Veille Réglementaire :

- ANSSI : recommandations sécurité applicables à la plateforme
- CNIL : guides RGPD
- OWASP : Top 10 vulnérabilités web (sources anglophones de référence)

B. Outils Utilisés

- Feedly : agrégateur RSS (50+ sources)
- Google Alerts : mots-clés stratégiques
- Twitter/X : experts tech à suivre
- Reddit (anglophone) : r/programming, r/reactjs, r/PostgreSQL, r/IOT
- Notion/Obsidian : prise de notes structurée

C. Fréquence et Organisation

- Veille quotidienne : 15-30 min (scan rapide)
- Veille hebdomadaire : 2h (analyse approfondie)
- Revue mensuelle : synthèse tendances majeures
- Partage équipe : présentation trimestrielle

II. COMPARAISON DES TECHNOLOGIES

✓ Critères RNCP Ce1.2.2 / A1.2 : Analyse comparative objective

Technologies évaluées (filière Développement)

A. Frameworks Frontend : React vs Vue.js vs Angular

Évaluation sur 5 critères notés de 0 à 10 (total /50). Notes appuyées sur les benchmarks publics (bundle size, Lighthouse) et les rapports State of JS / Stack Overflow Survey.

Critère 1 — Performance (rendu, bundle, optimisations) : React 9 | Vue 9 | Angular 7

Critère 2 — Courbe d'apprentissage / documentation : React 8 | Vue 9 | Angular 6

Critère 3 — Communauté (GitHub, NPM, emploi) : React 10 | Vue 8 | Angular 8

Critère 4 — Écosystème (librairies, tooling, IDE) : React 10 | Vue 8 | Angular 9

Critère 5 — Maintenance (cadence, breaking changes) : React 9 | Vue 8 | Angular 9

TOTAL : React 46/50 | Vue 42/50 | Angular 39/50

CHOIX RECOMMANDÉ : React 18 + TypeScript. React domine sur la communauté et l'écosystème (viviers de développeurs et de librairies les plus larges du marché), facilitant le recrutement et la maintenance long terme. Combiné à TypeScript, il offre un typage strict réduisant les bugs runtime, et s'intègre nativement avec Leaflet (cartographie) et Socket.io (temps réel) requis par ECOTRACK.

B. Frameworks Backend : Node.js/Express vs FastAPI vs Spring Boot

Évaluation sur 5 critères notés de 0 à 10 (total /50). Notes appuyées sur les benchmarks TechEmpower et la documentation officielle des frameworks.

Critère 1 — Performance (req/s, latence) : Node/Express 8 | FastAPI 9 | Spring Boot 8

Critère 2 — Scalabilité (microservices, conteneurs) : Node/Express 9 | FastAPI 9 | Spring Boot 9

Critère 3 — Typage / robustesse : Node/Express 6 | FastAPI 9 | Spring Boot 10

Critère 4 — Async / temps réel (WebSocket, MQTT) : Node/Express 10
| FastAPI 8 | Spring Boot 7

Critère 5 — Rapidité de développement / déploiement : Node/Express 9
| FastAPI 9 | Spring Boot 6

TOTAL : Node/Express 42/50 | FastAPI 44/50 | Spring Boot 40/50

CHOIX RECOMMANDÉ : approche hybride FastAPI (Python) + Node.js. FastAPI est retenu pour l'API métier et le moteur d'optimisation TSP : typage Pydantic, documentation OpenAPI automatique et écosystème Python scientifique (NetworkX, OR-Tools) idéal pour l'algorithme du voyageur de commerce. Node.js est retenu pour les services temps réel (ingestion MQTT, notifications, WebSocket Socket.io) où sa boucle d'événements non bloquante excelle face aux ~500 000 mesures/jour. Spring Boot, bien que très robuste, est écarté pour sa lourdeur de mise en œuvre et son éloignement de la stack Python/JS de l'équipe.

C. Bases de Données : PostgreSQL vs MongoDB vs TimescaleDB

Évaluation sur 5 critères notés de 0 à 10 (total /50), focalisée sur les cas d'usage ECOTRACK (géospatial des conteneurs et tournées, séries temporelles des mesures de niveau).

Critère 1 — Performance (charge d'écriture IoT) : PostgreSQL 8 | MongoDB 8 | TimescaleDB 10

Critère 2 — Scalabilité / volumétrie : PostgreSQL 8 | MongoDB 9 | TimescaleDB 9

Critère 3 — Requêtes complexes (jointures, SQL géospatial) : PostgreSQL 10 | MongoDB 6 | TimescaleDB 9

Critère 4 — Extensions (géospatial, séries temporelles) : PostgreSQL 10 | MongoDB 7 | TimescaleDB 10

Critère 5 — Coût / licence (open source) : PostgreSQL 10 | MongoDB 7 | TimescaleDB 9

TOTAL : PostgreSQL 46/50 | MongoDB 37/50 | TimescaleDB 47/50

CHOIX RECOMMANDÉ : PostgreSQL 15 avec l'extension PostGIS comme base de données principale (relationnel, géospatial pour conteneurs et tournées, requêtes complexes), complété par TimescaleDB (extension PostgreSQL) pour les séries temporelles des ~500 000 mesures/jour. Cette combinaison conserve un moteur unique et un dialecte SQL homogène — TimescaleDB étant une extension de

PostgreSQL, l'intégration PostGIS + TimescaleDB se fait sans rupture technologique. MongoDB, malgré sa souplesse, est écarté : la modélisation géospatiale et les requêtes analytiques complexes y sont moins matures que sous PostgreSQL/PostGIS.

III. TENDANCES ET INNOVATIONS 2025

A. Tendances Globales (Toutes Filières)

1. IA Générative et LLMs

- Impact : Assistants intelligents pour chauffeurs, génération rapports automatisés
- Technologies : GPT-4, Claude, LLaMA, fine-tuning
- Application ECOTRACK : Chatbot support chauffeurs, analyse prédictive textuelle

2. Edge Computing et IoT

- Impact : Traitement local sur capteurs, réduction latence
- Technologies : Edge AI, TensorFlow Lite, MQTT 5.0
- Application ECOTRACK : Pré-traitement données capteurs, alertes offline

3. Zero Trust Security

- Impact : Sécurisation granulaire des accès, conformité renforcée
- Technologies : Authentification continue, micro-segmentation
- Application ECOTRACK : Accès chauffeurs/gestionnaires par rôles stricts

B. Tendances Spécifiques par Filière

Tendances 2025 pertinentes pour ECOTRACK (Développement)

1. React Server Components (RSC) : Rendu hybride client/serveur
→ Impact ECOTRACK : Performance améliorée, SEO optimisé
2. WebAssembly (WASM) : Code natif dans navigateur
→ Impact : Algorithmes lourds (TSP) exécutés côté client
3. TypeScript 5+ : Typage encore plus strict
→ Impact : Réduction bugs runtime, maintenabilité
4. Micro-Frontends : Architecture modulaire frontend
→ Impact : Équipes autonomes, déploiement indépendant modules

IV. CHOIX TECHNOLOGIQUES FINAUX POUR ECOTRACK



Critères RNCP Ce1.2.3 / A1.2 : Choix justifiés et documentés

A. Synthèse des Choix par Composant

—
COMPOSANT : Frontend

TECHNOLOGIE RETENUE : React 18 + TypeScript

SCORE COMPARATIF : 46/50

JUSTIFICATION :

1. Plus large communauté et écosystème du marché (recrutement et maintenance facilités).
2. Typage strict TypeScript réduisant les bugs runtime sur une application métier complexe.
3. Intégration native avec Leaflet (cartographie) et Socket.io (temps réel) requis par ECOTRACK.

RISQUES : verbosité et configuration initiale ; mitigés par l'usage de Vite et d'une base de composants standardisée.

—
COMPOSANT : Backend

TECHNOLOGIE RETENUE : FastAPI (API métier + optimisation TSP) + Node.js (services temps réel)

SCORE COMPARATIF : 44/50 (FastAPI) / 42/50 (Node.js)

JUSTIFICATION :

1. FastAPI : typage Pydantic, doc OpenAPI auto et écosystème Python (OR-Tools, NetworkX) pour le TSP.
2. Node.js : boucle d'événements non bloquante optimale pour MQTT, WebSocket et notifications temps réel.

3. Découpage en 5 microservices (IoT, API, Optim, Auth, Notif) permettant un scaling indépendant.

RISQUES : surcoût d'exploitation de deux runtimes ; mitigé par la conteneurisation Docker et un découpage clair des responsabilités.

—
COMPOSANT : Base de données

TECHNOLOGIE RETENUE : PostgreSQL 15 + PostGIS + TimescaleDB

SCORE COMPARATIF : 47/50

JUSTIFICATION :

1. PostGIS couvre le géospatial (positions des conteneurs, calcul de tournées) en SQL natif.

2. TimescaleDB absorbe les ~500 000 mesures/jour avec compression et agrégats continus performants.

3. Moteur unique PostgreSQL : dialecte SQL homogène, une seule compétence d'exploitation, open source.

RISQUES : volumétrie des séries temporelles ; mitigée par les politiques de rétention et la compression TimescaleDB.

B. Stack Technique Complète Recommandée

Frontend : React 18 + TypeScript, Vite, Leaflet + OpenStreetMap, Recharts, Socket.io client

Backend : FastAPI (Python) pour l'API métier et l'optimisation TSP, Node.js pour les services temps réel ; architecture microservices (IoT, API, Optim, Auth, Notif)

Base de données : PostgreSQL 15 + PostGIS (géospatial) + TimescaleDB (séries temporelles) ; Redis (cache et files)

Infrastructure : Docker (conteneurisation), Nginx (reverse proxy), MQTT Mosquitto (broker IoT), WebSocket (Socket.io)

Monitoring : Prometheus + Grafana (métriques), alerting sur CVE et breaking changes (Dependabot / Renovate)

Sécurité : authentification JWT, 2FA TOTP, contrôle d'accès par rôles (RBAC), chiffrement TLS

CI/CD : GitHub Actions (build, tests, image Docker), déploiement automatisé, méthodologie Scrum (8 sprints de 2 semaines)

V. PLAN DE VEILLE CONTINUE

A. Pendant le Projet (16 semaines)

- Veille hebdomadaire : 1h par développeur (technologies utilisées)
- Sprint Review : partage découvertes et bonnes pratiques
- Tech Watch : présentation mensuelle tendances équipe
- Alertes critiques : CVE sécurité, breaking changes librairies

B. Post-Déploiement (Maintenance)

- Monitoring versions : Dependabot + Renovate Bot
- Veille sécurité : ANSSI, OWASP, CVE databases
- Roadmaps officielles (anglophones) : React, FastAPI, PostgreSQL, TimescaleDB
- Participation communauté : Conférences, meetups, contributions open-source

C. Ressources Recommandées

- Newsletters (anglophones) : JavaScript Weekly, Python Weekly, Postgres Weekly, Node Weekly
- Podcasts (anglophones) : Syntax.fm, Talk Python To Me, Software Engineering Daily
- YouTube (anglophones) : Fireship, Hussein Nasser, ArjanCodes
- Formation continue : Udemy, Pluralsight, Cloud certifications

FIN DE LA VEILLE TECHNOLOGIQUE

PROJET ECOTRACK

ARCHITECTURE LOGICIELLE

Mastère EADL — Filière Développement Logiciel (RNCP 38822)

I. VISION GLOBALE DE L'ARCHITECTURE

✓ Critères RNCP Ce1.3.1 / A2.1 : Architecture complète et cohérente

Figure 1 — Schéma d'architecture global d'ECOTRACK (voir Annexe A).

Le schéma global d'ECOTRACK se lit de gauche à droite en quatre plans. À gauche, le parc de 2 000 conteneurs connectés, répartis sur 12 secteurs d'une métropole de 500 000 habitants, publie une mesure de remplissage toutes les 15 minutes (soit environ 500 000 messages par jour) vers un broker MQTT Mosquitto, qui alimente le bus événementiel Apache Kafka. Au centre, une API Gateway assure la façade unique : terminaison TLS, authentification, rate limiting et routage vers une grappe de microservices conteneurisés, organisés en Bounded Contexts (Ingestion IoT, Optimisation des tournées, Facturation, Gamification, Signalement citoyen, Authentification). Ces services partagent une couche de persistance PostgreSQL 15 enrichie de PostGIS (géospatial) et un cache Redis. À droite, trois applications clientes — le tableau de bord web React des gestionnaires et administrateurs, l'application mobile Citoyen et l'application mobile Agent (React Native) — consomment les API via la Gateway et reçoivent les mises à jour temps réel. L'ensemble est containerisé avec Docker, orchestré par Docker Compose en développement et ciblant Kubernetes en M2.

ECOTRACK repose sur une architecture microservices structurée par le Domain-Driven Design. Plutôt qu'un monolithe, le système est découpé en Bounded Contexts qui correspondent aux domaines métier réels du projet : l'Ingestion IoT (réception et validation des mesures capteurs), l'Optimisation des tournées (calcul d'itinéraires TSP/2-opt), la Facturation (suivi des collectes et coûts), la Gamification (points, badges, défis, classements — module central côté citoyen), le Signalement citoyen (remontées terrain géolocalisées) et l'Authentification (gestion des comptes et des rôles). Ce découpage répond aux trois contraintes structurantes du projet : un fort volume de données temps réel (ingestion MQTT/Kafka continue), des traitements hétérogènes (CRUD métier, calcul d'optimisation intensif, requêtes géospatiales) et un besoin de scalabilité indépendante par domaine. On peut ainsi monter en charge le seul service réellement sollicité —

typiquement l'Ingestion IoT ou l'Optimisation — sans surdimensionner l'ensemble.

En façade, une API Gateway joue le rôle de point d'entrée unique : elle centralise la terminaison TLS, la validation des jetons JWT, le contrôle RBAC, le rate limiting (appuyé sur Redis) et le routage vers le bon microservice. Derrière la Gateway, la communication inter-services est majoritairement asynchrone et événementielle, portée par le bus Apache Kafka : les producteurs publient des événements métier (mesure reçue, seuil franchi, signalement créé, tournée planifiée, points attribués) sur des topics dédiés, et chaque service consomme les topics qui le concernent. Ce modèle découple fortement les services, absorbe les pics de charge IoT et permet de rejouer les flux. La persistance s'appuie sur PostgreSQL 15 + PostGIS (données métier et géospatiales, index GiST) et sur Redis (cache, sessions, compteurs de rate limiting). Tous les composants sont containerisés sous Docker ; l'orchestration Kubernetes constitue la cible de la phase M2.

Les interactions suivent un flux clair. Les capteurs des 2 000 conteneurs émettent une mesure toutes les 15 minutes vers le broker MQTT, qui la dépose sur le topic Kafka d'ingestion ; le service Ingestion IoT consomme ce flux, valide et persiste les mesures, puis publie un événement d'alerte lorsqu'un seuil de remplissage est franchi. Le tableau de bord des gestionnaires reçoit ces événements en temps réel et met à jour la cartographie. Lorsqu'un gestionnaire planifie une collecte, le service Optimisation calcule l'itinéraire le plus court (algorithme TSP / 2-opt), persiste la tournée et publie un événement consommé par l'application mobile Agent. Le service Signalement citoyen reçoit les remontées de l'application mobile Citoyen, crédite des points via le service Gamification et notifie le gestionnaire de zone.

L'Authentification, partagée par toutes les applications, gère les quatre rôles RBAC, garantissant une séparation nette des responsabilités.

A. Principes Architecturaux

- Modularité (DDD) : un microservice par Bounded Context, chacun maître de son schéma de données et déployable indépendamment, ce qui isole les évolutions et les pannes par domaine.

- Scalabilité horizontale : chaque service est sans état et répliquable derrière l'API Gateway ; le bus Kafka et le cache Redis absorbent les pics, l'Ingestion IoT et l'Optimisation montant en charge indépendamment du reste.
- Résilience : communication asynchrone par événements (Kafka), pattern Circuit Breaker sur les appels synchrones inter-services et dégradation gracieuse (réponses en cache, files de rattrapage) pour viser un uptime de 99,5 %.
- Sécurité applicative : façade unique via l'API Gateway (TLS, JWT, RBAC 4 rôles, rate limiting), moindre privilège entre services et journalisation des accès (la sécurité réseau/infra relève des filières Cyber et Data).
- Maintenabilité : contrats d'API versionnés et documentés (OpenAPI/Swagger), tests automatisés (unitaires, intégration, contrat, e2e) et conteneurs Docker reproductibles d'un environnement à l'autre.

B. Vue d'Ensemble des Composants

- Couche Présentation : 3 applications clientes — Dashboard web React (gestionnaires/admins), App mobile Citoyen et App mobile Agent (React Native).
- Couche Métier : API Gateway en façade + microservices DDD (Ingestion IoT, Optimisation, Facturation, Gamification, Signalement, Authentification) communiquant via Kafka.
- Couche Données : PostgreSQL 15 + PostGIS (métier et géospatial, index GiST), Redis (cache/sessions/rate limiting), partitionnement temporel des mesures.
- Couche Infrastructure : containerisation Docker, bus événementiel Kafka, orchestration Kubernetes en cible M2 (le réseau et la supervision infra relèvent des filières Cyber et Data).

II. ARCHITECTURE DÉTAILLÉE PAR FILIÈRE

Architecture Microservices orientée Domain-Driven Design (Filière Développement)

A. Couche Présentation — 3 applications clientes

1. Dashboard web (gestionnaires et administrateurs)

- Framework : React 18 avec TypeScript
- State Management : Redux Toolkit + RTK Query
- Routing : React Router v6
- UI Library : Material-UI (MUI) + Tailwind CSS
- Cartographie : Leaflet + React-Leaflet (fonds OpenStreetMap)
- Temps réel : Socket.io-client pour WebSocket
- Charts : Recharts pour visualisations

2. Application mobile Citoyen (React Native)

- Framework : React Native
- Fonctionnalités : Scan QR codes, Géolocalisation, Mode offline

B. Couche Métier — API Gateway + microservices DDD

L'API Gateway constitue la façade unique du système : terminaison TLS, vérification du JWT, contrôle RBAC, rate limiting (Redis) et routage vers le bon microservice. Derrière elle, six microservices correspondant aux Bounded Contexts du domaine communiquent de façon asynchrone via le bus Apache Kafka (producteurs/consommateurs sur topics dédiés) et, lorsque nécessaire, par appels HTTP/gRPC internes protégés par Circuit Breaker :

1. Service Ingestion IoT (Bounded Context Ingestion IoT, Node.js)

- Ingestion MQTT : 2 000 capteurs, 1 mesure toutes les 15 min (~500 000 messages/jour), MQTT Mosquitto → topic Kafka
- Validation données : Schéma JSON
- Déclenchement alertes : Seuils configurables

2. Service Cœur métier / API REST (Python FastAPI)

- Endpoints CRUD : Conteneurs, Tournées, Alertes, Utilisateurs
- Documentation : OpenAPI/Swagger
- Validation : Pydantic models
- Rate limiting : 1000 req/min par user

3. Service Optimisation des tournées (Bounded Context Optimisation, Python)

- Algorithme TSP : Génétique OU glouton

- Contraintes : capacité véhicule, horaires, zones
 - Performance : <30s pour tournée 50 conteneurs
4. Service Authentification (Bounded Context Authentication, Node.js)
- JWT tokens : Access (15 min) + Refresh (7 jours)
 - 2FA : TOTP (Google Authenticator)
- RBAC : 4 rôles (Citoyen, Agent, Gestionnaire, Administrateur), principe de moindre privilège
5. Service Gamification + Signalement (Bounded Contexts Gamification et Signalement citoyen, Node.js)
- Email : Nodemailer + SendGrid
 - Signalement citoyen : remontées géolocalisées (déduplication <1 h), crédit de points, notification au gestionnaire de zone
 - Push notifications : Firebase Cloud Messaging
- C. Couche Données — persistance polyglotte légère
1. PostgreSQL 15 + PostGIS (base principale)
- Tables : users, containers, measurements, routes, alerts
 - Index géospatiaux : GiST pour requêtes spatiales rapides
 - Partitionnement : measurements par mois
2. Redis (cache et sessions)
- Cache API responses : TTL 5-60 min
 - Sessions utilisateurs
 - Rate limiting counters
3. Apache Kafka (bus événementiel)
- Séries temporelles mesures IoT
 - Agrégations automatiques (1h, 1j, 1 semaine)
 - Compression données anciennes
- D. Patterns de Conception Utilisés
- Repository : chaque microservice isole son accès aux données derrière une interface de dépôt (ex. ConteneurRepository, MeasureRepository). La logique métier ignore le détail SQL/PostGIS, ce qui facilite les tests (dépôts simulés) et l'évolution du schéma sans impacter les couches supérieures.
 - Strategy : le service Optimisation expose une interface StrategieOptimisation implémentée par plusieurs algorithmes TSP (gloutonne pour le MVP, 2-opt et génétique en évolution). La stratégie est sélectionnée à l'exécution selon la taille de la tournée et le budget

temps, sans modifier le code appelant — clé pour faire évoluer la qualité d'optimisation sans régression.

- Observer / event-driven (Kafka) : les services publient des événements métier sur des topics ; les consommateurs intéressés y réagissent de façon découplée. Côté clients, le dashboard s'abonne aux flux temps réel (remplissage, alertes) poussés via WebSocket, sans polling, maintenant la carte à jour.
- API Gateway : point d'entrée unique en façade des microservices ; centralise TLS, authentification JWT, contrôle RBAC, rate limiting et routage, et masque la topologie interne aux applications clientes.
- Circuit Breaker (résilience, ex. Resilience4j) sur les appels synchrones inter-services pour éviter les pannes en cascade ; CQRS possible côté lecture analytics (séparation des modèles écriture/lecture pour les KPIs gestionnaire). Les préoccupations transverses (auth, journalisation, gestion d'erreurs) sont traitées en middlewares chaînés à l'API Gateway et dans chaque service.

III. SCHÉMAS D'ARCHITECTURE

Schémas Obligatoires :

1. Architecture Globale : Vue d'ensemble de tous les composants
2. Flux de Données : Parcours données de l'ingestion à l'affichage
3. Architecture de déploiement : conteneurs Docker, API Gateway et cible Kubernetes (M2)
4. Architecture Déploiement : Environnements (dev, staging, prod)
5. Schéma Base de Données : ERD avec relations
6. Diagramme de flux événementiel : topics Kafka, producteurs et consommateurs

Figure 1 — Architecture globale : vue d'ensemble des composants (capteurs MQTT, bus Kafka, API Gateway, microservices DDD, données, 3 applications clientes) — voir Annexe A.

Figure 2 — Flux de données temps réel : parcours d'une mesure du capteur jusqu'à l'affichage et l'alerte (capteur → MQTT → Kafka → service Ingestion IoT → PostgreSQL → WebSocket → dashboard) — voir Annexe B.

Figure 3 — Architecture de déploiement : conteneurs Docker, API Gateway, environnements dev / staging / production et cible d'orchestration Kubernetes (M2) — voir Annexe C.

Figure 4 — Modèle de données (ERD) : entités Utilisateur, Conteneur, Zone, Mesure, Tournée, Signalement, Alerte et Gamification (Point, Badge, Défi, Classement), avec index GiST PostGIS et partitionnement temporel des mesures — voir Annexe D.

Figure 5 — Diagramme de flux événementiel : topics Kafka (iot.measurements, alerts.raised, signals.created, routes.planned, gamification.points) et leurs producteurs/consommateurs — voir Annexe E.

Ces schémas, réalisés sous draw.io et PlantUML (C4 niveau 2), sont fournis en annexe au format vectoriel pour préserver leur lisibilité à l'impression.

IV. SCALABILITÉ ET PERFORMANCE

A. Stratégies de Scalabilité

1. Scalabilité Horizontale

- Load balancers : Distribution trafic sur N instances
- Microservices : scaling indépendant par Bounded Context derrière l'API Gateway
- BDD : réplicas en lecture PostgreSQL pour l'analytics ; cache Redis (hit >80 %)
- Bus Kafka : partitionnement des topics pour absorber les ~500 000 messages IoT/jour

2. Scalabilité Verticale

- Ressources : CPU, RAM, IOPS ajustables
- Cas d'usage : BDD principale, Spark master
- Limites : Plafond matériel (→ passer horizontal)

B. Objectifs de Performance

- Temps de réponse API : < 500 ms au 95e centile (p95)
- Requêtes SQL : < 200 ms pour 90 % des requêtes
- Charge : 500 utilisateurs simultanés supportés
- Cache : taux de hit Redis > 80 % sur les lectures fréquentes
- Disponibilité : uptime 99,5 % ($\approx$ 43 h d'indisponibilité/an au maximum)

C. Tests de Charge Prévus

- Outils : Apache JMeter, k6, Locust
- Scénarios : montée jusqu'à 500 utilisateurs simultanés (cible NFR), avec marge
- Fréquence : Sprint 3, 6, 8 (+ avant prod)
- Métriques : Throughput, latency, error rate

V. SÉCURITÉ ET CONFORMITÉ

✓ Critères RNCP Ce1.3.2 / A2.1 : Sécurité intégrée

A. Authentification et Autorisation

- Authentification : JWT (Access 15 min + Refresh 7j)
- MFA : TOTP obligatoire pour gestionnaires et administrateurs ; blocage après 5 tentatives
- RBAC : 4 rôles (Citoyen, Agent, Gestionnaire, Administrateur), principe de moindre privilège
- Sessions : Redis avec TTL
- Password : bcrypt (cost 12), min 12 caractères

B. Chiffrement

- Transport : TLS 1.3 obligatoire (HTTPS)
- At-rest : AES-256 pour données sensibles (BDD)
- Secrets : Vault (HashiCorp) ou AWS Secrets Manager
- Certificats : Let's Encrypt (auto-renew)

C. Conformité RGPD

- Consentement : Opt-in explicite utilisateurs
- Droit accès : API GET /me/data
- Droit effacement : Soft delete + purge 90j
- Portabilité : Export JSON/CSV
- Minimisation : Données strictement nécessaires
- DPO : Désigné + registre traitements

D. Recommandations ANSSI

- Guide Sécurité Cloud : Chiffrement, cloisonnement
- Bonnes pratiques DevSecOps : Scan vulnérabilités
- Journalisation : Logs 1 an, audit trail

FIN DE L'ARCHITECTURE LOGICIELLE

PROJET ECOTRACK

MODÉLISATION

MODÉLISATION UML

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822) — Compétence C2.1

MODÉLISATION UML

Cette section présente la modélisation UML de la plateforme ECOTRACK pour le périmètre applicatif de la filière Développement : diagramme de cas d'utilisation, diagramme de classes et diagrammes de séquence. Les diagrammes graphiques sont fournis en annexe ; chaque figure est ici accompagnée d'une description détaillée. La modélisation s'appuie directement sur les scénarios du cahier des charges (section IV) et couvre les cinq acteurs du système : Citoyen, Agent de collecte, Gestionnaire, Administrateur et le Système IoT (capteurs des conteneurs).

A. Diagramme de Cas d'Utilisation

Figure 1 — Diagramme de cas d'utilisation ECOTRACK (voir Annexe)

Le système ECOTRACK met en scène cinq acteurs. Quatre sont des acteurs humains, le cinquième est un acteur système non humain :

- Citoyen (15 000 utilisateurs actifs) — acteur principal de la dimension participative et de la gamification, via l'application mobile citoyen.
- Agent de collecte (50) — opérateur terrain, via l'application mobile agent.
- Gestionnaire (10) — supervise l'exploitation et planifie les tournées via le dashboard web.
- Administrateur (3) — administre la plateforme (utilisateurs, paramétrage, gamification) via le dashboard web.
- Système IoT — acteur secondaire représentant les capteurs des 2 000 conteneurs, qui alimentent la plateforme en mesures de remplissage.

Cas d'utilisation du Citoyen :

- UC-C01 — Signaler un conteneur plein : depuis la carte, le citoyen localise un conteneur, choisit le type de problème (« conteneur plein »), ajoute une photo optionnelle et valide. Le signalement est horodaté et géolocalisé ; le citoyen reçoit +10 points de gamification (relation « include » vers Crédit de points) et une notification part vers le gestionnaire de zone. Exception : doublon (< 1 h), conteneur inexistant.
- UC-C02 — Consulter son historique et son impact : statistiques personnelles (signalements, points, badges), impact environnemental (CO₂ économisé) et position dans le classement.
- UC-C03 — Participer à un défi collectif : le citoyen s'inscrit à un défi (ex. « Zéro débordement — Quartier Centre »), effectue des signalements pendant la période, puis les résultats et récompenses sont calculés à la clôture.

Cas d'utilisation de l'Agent de collecte :

- UC-A01 — Recevoir la tournée du jour : après authentification mobile, l'agent visualise sa tournée optimisée (itinéraire, étapes numérotées, nombre de conteneurs, distance, durée estimée) puis démarre la tournée, activant le tracking.

- UC-A02 — Valider une collecte : à chaque arrêt, l'agent scanne le QR code du conteneur (ou sélection manuelle), confirme la collecte avec le volume ; le système enregistre timestamp, agent, volume et position GPS, puis affiche l'étape suivante.
- UC-A03 — Signaler une anomalie : l'agent déclare une anomalie terrain (conteneur endommagé, accès bloqué), choisit le type, ajoute commentaire et/ou photo ; le système enregistre et alerte le gestionnaire.

Cas d'utilisation du Gestionnaire :

- UC-G01 — Créer une tournée optimisée : le gestionnaire sélectionne une zone et un seuil de remplissage (ex. > 70 %) ; le système propose les conteneurs éligibles puis lance l'optimisation algorithmique (TSP avec amélioration 2-opt) qui calcule l'itinéraire optimal. Le gestionnaire ajuste si besoin, valide et assigne la tournée à un agent (« include » UC-A01 côté agent).
- UC-G02 — Monitoring temps réel : dashboard cartographique où les conteneurs sont colorés par état, identification des conteneurs critiques (> 90 %), réception des alertes en temps réel et déclenchement éventuel d'une collecte d'urgence.
- UC-G03 — Générer un rapport mensuel : sélection d'une période et de KPIs (collectes, distances, débordements), génération d'un rapport PDF structuré avec graphiques, téléchargement ou envoi par email.

Cas d'utilisation de l'Administrateur :

- UC-AD01 — Gérer les utilisateurs : depuis le backoffice, l'administrateur consulte, crée, modifie ou désactive des comptes et assigne rôles et permissions (RBAC). Toutes les actions sont tracées dans les logs d'audit.
- UC-AD02 — Configurer les alertes : définition des seuils d'alerte par type de conteneur, des destinataires des notifications et activation/désactivation des canaux (email, SMS, push).

Cas d'utilisation transversaux :

- UC-T01 — Authentification sécurisée : tout acteur humain s'authentifie par email et mot de passe ; pour le Gestionnaire et l'Administrateur, une double authentification (MFA / code TOTP) est exigée. En cas de succès, un JWT à durée limitée est émis. Blocage du compte après 5 tentatives échouées. Ce cas est en relation « include » avec l'ensemble des cas des acteurs humains.
- UC-T02 — Réception de données IoT : porté par l'acteur Système IoT. Chaque capteur mesure le niveau de remplissage (ultrason) et publie via MQTT (container_id, fill_level, temperature, timestamp), toutes les 15 minutes ou sur changement significatif. Le broker authentifie le message, les données transitent par Kafka puis PostgreSQL, et le dashboard se met à jour en temps réel. Ce cas « extend » la mise à jour de la carte et le déclenchement automatique d'alertes lorsque les seuils sont franchis.

B. Diagramme de Classes

Figure 2 — Diagramme de classes ECOTRACK (voir Annexe)

Le diagramme de classes décrit le domaine métier d'ECOTRACK. On distingue le noyau exploitation (Utilisateur, Conteneur, Zone, Mesure, Tournée, Signalement, Alerte) et le module central de gamification (Point, Badge, Défi, Classement).

Cœur du domaine :

- Utilisateur — attributs : id, nom, prenom, email, motDePasseHash, role, secretMFA, mfaActive, dateCreation, statut. Méthodes : sAuthentifier(), verifierMFA(), changerMotDePasse(). La classe Utilisateur est spécialisée (héritage) en quatre sous-types selon le rôle : Citoyen, Agent, Gestionnaire, Administrateur — chacun portant ses comportements propres (ex. Citoyen.totalPoints(), Agent.tourneeDuJour(), Gestionnaire.creerTournee(), Administrateur.gererUtilisateurs()).
- Conteneur — attributs : id, codeQR, type, capacite, latitude, longitude, niveauActuel, statut. Méthodes : estPlein(), estCritique(), derniereMesure(), historiqueMesures(). Le Conteneur appartient à une Zone et est suivi par le Système IoT.
- Zone — attributs : id, nom, secteur, geometrie (polygone PostGIS), gestionnaireResponsable. Méthodes : conteneurs(), tauxRemplissageMoyen(). Découpage des 12 secteurs géographiques.
- Mesure — attributs : id, dateHeure, niveau, temperature, qualiteSignal, source. Méthode : estAuDessusSeuil(seuil). Donnée unitaire remontée par le capteur d'un Conteneur.
- Tournée — attributs : id, date, statut, distanceTotale, dureeEstimee, seuilDeclenchement. Méthodes : optimiser() (TSP / 2-opt), ajouterArret(), genererFeuilleDeRoute(), demarrer(), cloturer(). Résultat de l'optimisation algorithmique, assignée à un Agent. La séquence des arrêts est portée par une classe d'association Arret (ordre, heurePrevue, heureReelle, volumeCollecte, statutCollecte).
- Signalement — attributs : id, type (conteneurPlein, anomalie...), dateHeure, latitude, longitude, photoUrl, commentaire, statut. Méthodes : enregistrer(), estDoublon(). Émis par un Citoyen (UC-C01) ou un Agent (UC-A03) à propos d'un Conteneur.
- Alerte — attributs : id, type, niveauGravite, dateCreation, statut, canalNotification. Méthodes : declencher(), acquitter(), escalader(). Naît du dépassement d'un seuil sur un Conteneur (UC-T02) ou d'un Signalement, et notifie le Gestionnaire (UC-G02).

Module de gamification (central) :

- Point — attributs : id, valeur, motif (ex. signalement validé), dateAttribution. Méthode : crediter(). Chaque action éligible d'un Citoyen génère des Points.
- Badge — attributs : id, libelle, description, iconeUrl, conditionObtention. Méthode : estDebloque(citoyen). Plus de 30 badges récompensant la progression.
- Défi — attributs : id, titre, description, dateDebut, dateFin, objectif, recompense, statut. Méthodes : inscrire(citoyen), calculerResultats(), attribuerRecompenses(). Support des défis collectifs (UC-C03).
- Classement — attributs : id, periode, portee (globale / par zone), dateCalcul. Méthodes : calculerRang(citoyen), top(n). Leaderboard consultable par les Citoyens (UC-C02).

Relations et cardinalités :

- Utilisateur <|-- Citoyen, Agent, Gestionnaire, Administrateur (héritage / spécialisation par rôle).
 - Zone 1 -- 0..* Conteneur (une Zone regroupe plusieurs Conteneurs).
 - Conteneur 1 -- 0..* Mesure (un Conteneur produit de nombreuses Mesures dans le temps).
 - Conteneur 1 -- 0..* Alerte et Conteneur 1 -- 0..* Signalement.
 - Tournée 0..* -- 1..* Conteneur via la classe d'association Arret (un Conteneur peut figurer dans plusieurs Tournées).
 - Agent 1 -- 0..* Tournée (un Agent réalise plusieurs Tournées ; une Tournée est assignée à un Agent).
 - Citoyen 1 -- 0..* Signalement et Citoyen 1 -- 0..* Point.
 - Citoyen 0..* -- 0..* Badge (un Citoyen débloque plusieurs Badges, un Badge est partagé par plusieurs Citoyens).
 - Citoyen 0..* -- 0..* Défi (inscription aux défis) et Défi 1 -- 0..1 Classement de défi.
- Cette structure orientée objet sépare clairement les données mesurées (Mesure), les entités physiques (Conteneur, Zone), le pilotage opérationnel (Tournée, Arret, Alerte, Signalement), la gestion des accès (Utilisateur et ses rôles) et l'engagement citoyen (gamification), ce qui facilite le découpage en services et le mapping objet-relationnel vers PostgreSQL/PostGIS.

C. Diagrammes de Séquences

Figure 3 — Séquence « Signalement citoyen d'un conteneur plein » (voir Annexe)

Ce scénario (UC-C01) illustre l'enchaînement entre l'application mobile citoyen, l'API REST, le module de gamification et le mécanisme d'alerte. Participants : Citoyen, App mobile citoyen, API/Service Signalement, Service Gamification, Service Alertes, Gestionnaire.

1. Le Citoyen ouvre l'application et localise le conteneur sur la carte (GPS).
2. Il sélectionne « Signaler un problème », choisit le type « Conteneur plein » et ajoute une photo optionnelle, puis valide.
3. L'application envoie une requête POST /signalements (container_id, type, position, photo) à l'API, accompagnée du JWT.
4. Le Service Signalement vérifie l'absence de doublon récent (estDoublon(), < 1 h) et que le conteneur existe, puis crée et persiste le Signalement horodaté et géolocalisé.
5. En cas de succès, le Service Signalement appelle de façon asynchrone le Service Gamification : crediter(+10 points) au profil du Citoyen, mise à jour des badges et du classement.
6. En parallèle, le Service Signalement notifie le Service Alertes, qui rattache l'information au Conteneur concerné et notifie le Gestionnaire de zone (alerte temps réel via WebSocket / push).
7. L'API renvoie la confirmation à l'application ; le Citoyen voit son nouveau total de points et l'accusé de signalement.

Flux alternatif : si l'étape 4 détecte un doublon ou un conteneur inexistant, l'API renvoie une erreur 409 / 404, aucun point n'est crédité et aucune alerte n'est créée.

Figure 4 — Séquence « Réception d'une mesure IoT + déclenchement d'alerte » (voir Annexe)

Ce scénario (UC-T02) décrit la chaîne d'ingestion temps réel des mesures de remplissage.

Participants : Capteur (Système IoT), Broker MQTT (Mosquitto, TLS), Service d'ingestion, Kafka, Service de traitement, Base PostgreSQL/PostGIS, Dashboard (carte temps réel), Service Alertes, Gestionnaire.

1. Le Capteur d'un Conteneur mesure le niveau (ultrason) toutes les 15 minutes ou sur changement significatif, et publie un message MQTT (container_id, fill_level, temperature, timestamp).
2. Le Broker MQTT authentifie le capteur (certificat TLS) et relaie le message.
3. Le Service d'ingestion consomme le message, le valide et le publie dans un topic Kafka (découplage et absorption de la charge, ~400 messages/minute).
4. Le Service de traitement consomme l'événement Kafka, crée un objet Mesure et le persiste dans PostgreSQL/PostGIS, associé au Conteneur.
5. Le service met à jour le niveauActuel du Conteneur et pousse la mise à jour vers le Dashboard (WebSocket) : le marqueur du conteneur change de couleur sur la carte temps réel.
6. Le service évalue le seuil via estAuDessusSeuil() / estCritique(). Si le seuil configuré (UC-AD02, ex. > 90 %) est franchi, il invoque le Service Alertes : declencher() crée une Alerte rattachée au Conteneur.
7. Le Service Alertes notifie le Gestionnaire en temps réel via le canal configuré (push / email / SMS). Le Gestionnaire peut alors acquitter() l'alerte ou déclencher une collecte d'urgence (UC-G02).

Flux alternatif : si le seuil n'est pas atteint, le flux s'arrête après la mise à jour de la carte (étape 5), sans création d'alerte.

CONCLUSION

Le choix d'UML s'impose pour ECOTRACK côté Développement car la plateforme est avant tout une application orientée objet, structurée en services (authentification, ingestion IoT, optimisation des tournées, gamification, gestion des alertes, cartographie) et déclinée en trois clients : application mobile citoyen, application mobile agent et dashboard web. UML permet de décrire simultanément le périmètre fonctionnel par acteur (cas d'utilisation), la structure statique du domaine (diagramme de classes, incluant le module de gamification) et le comportement dynamique inter-services (diagrammes de séquence), couvrant ainsi l'ensemble des besoins de conception logicielle du projet. BPMN aurait été pertinent pour décrire des processus métier organisationnels transverses, mais il ne modélise ni les objets, ni leurs attributs et méthodes, ni les contrats d'interface entre services : il reste trop centré « flux » pour guider l'implémentation d'un système objet et de ses API REST. MERISE se concentre, lui, sur le modèle de données relationnel (MCD / MLD / MPD) ; or ECOTRACK manipule des données hétérogènes — séries temporelles IoT, données géospatiales PostGIS, files de messages MQTT/Kafka, données de gamification — qui ne se réduisent pas à un schéma purement relationnel, et MERISE n'exprime ni les comportements ni les interactions entre composants. UML offre donc la meilleure correspondance avec une architecture orientée objet et orientée services : les classes se traduisent directement en entités du domaine et en modèles de données, les diagrammes de séquence documentent les échanges entre composants (MQTT, Kafka, API REST, WebSocket) et le diagramme de cas d'utilisation cadre le périmètre fonctionnel des cinq acteurs. Cette cohérence entre modèle et code accélère la conception, fiabilise les développements et facilite la maintenance évolutive de la plateforme.

PROJET ECOTRACK

PLANIFICATION AGILE

Mastère EADL — Filière Développement Logiciel (RNCP 38822)

I. CHOIX DE LA MÉTHODOLOGIE AGILE

✓ Critères RNCP : Ce1.4.1 / A1.4 - Méthodologie spécifique

Le projet ECOTRACK retient la méthodologie Scrum, conformément à l'approche préconisée par le cahier des charges. Le besoin reste partiellement à découvrir : les règles d'optimisation des tournées (TSP/2-opt), les seuils d'alerte des conteneurs, les mécaniques de gamification engageantes pour 15 000 citoyens et les usages terrain des agents (scan, mode hors-ligne) doivent être affinés au contact des utilisateurs. Les sprints courts de deux semaines permettent de livrer un incrément fonctionnel à chaque itération et d'ajuster le produit grâce au feedback des gestionnaires, des agents et des citoyens. Le Product Backlog priorisé en MoSCoW garantit que le cœur de valeur (carte temps réel, ingestion MQTT, tournées optimisées, signalement citoyen) est livré avant les fonctionnalités secondaires. Les revues et rétrospectives sécurisent qualité et cadence sur les 4 mois (16 semaines) du projet, tandis qu'une gestion itérative des risques techniques (tests de charge et de couverture dès les premiers sprints) maîtrise un système IoT temps réel à forte incertitude fonctionnelle.

A. Framework Scrum

Rôles : Product Owner, Scrum Master, Équipe Dev

Événements : Sprint Planning, Daily Stand-up, Sprint Review, Rétrospective

Artefacts : Product Backlog, Sprint Backlog, Incrément

B. Organisation en Sprints

- Durée d'un sprint : 2 semaines
- Cadençage : Kick-off (S1), Sprint 0 (S2-3), Sprints 1-5 puis Soutenance (S15-16)
- Durée totale projet : 4 mois (16 semaines, janvier à avril)

II. BACKLOG PRODUIT

A. Product Backlog (filière Développement) — format « En tant que... je veux... afin de... » / MoSCoW / Story Points

US-001 : En tant que Citoyen, je veux signaler un conteneur plein depuis l'application mobile, en ajoutant une photo, afin d'alerter le service de collecte et gagner des points | MUST | 8 SP

US-002 : En tant que Système/API, je veux ingérer les mesures des capteurs via MQTT (toutes les 15 min, ~500 000 messages/jour) afin de connaître l'état de remplissage des 2 000 conteneurs en continu | MUST | 8 SP

US-003 : En tant que Gestionnaire, je veux visualiser la carte des conteneurs colorés par niveau de remplissage en temps réel afin de superviser l'état du parc et repérer les conteneurs critiques (>90 %) | MUST | 8 SP

US-004 : En tant que Gestionnaire, je veux créer une tournée et lancer son optimisation (TSP + 2-opt, seuil de remplissage paramétrable) afin de minimiser les distances parcourues et les coûts | MUST | 13 SP

US-005 : En tant qu'Agent de collecte, je veux recevoir ma tournée optimisée du jour sur mobile (itinéraire, étapes numérotées, distance, durée) afin de suivre mon parcours sans perte de temps | MUST | 8 SP

US-006 : En tant qu'Agent de collecte, je veux valider une collecte en scannant le QR code du conteneur (volume, GPS, horodatage) afin de tracer précisément chaque passage | MUST | 8 SP

US-007 : En tant qu'Agent de collecte, je veux utiliser l'application en mode hors-ligne et synchroniser mes validations au retour de connexion afin de travailler dans les zones sans réseau | MUST | 8 SP

US-008 : En tant qu'Utilisateur, je veux m'authentifier via JWT, avec MFA pour les gestionnaires et administrateurs, afin de sécuriser l'accès à la plateforme | MUST | 5 SP

US-009 : En tant qu'Administrateur, je veux gérer les rôles et permissions (RBAC à 4 rôles : Citoyen, Agent, Gestionnaire, Admin) afin de cloisonner les accès selon le principe de moindre privilège | MUST | 5 SP

US-010 : En tant que Citoyen, je veux accumuler des points, débloquent des badges (30+) et suivre ma progression afin d'être récompensé pour mon engagement éco-responsable | MUST | 8 SP

US-011 : En tant que Citoyen, je veux participer à des défis collectifs et consulter un classement (leaderboard) afin de me motiver et de comparer mon impact à celui des autres | SHOULD | 5 SP

US-012 : En tant que Gestionnaire, je veux recevoir des alertes lorsqu'un conteneur dépasse un seuil de remplissage afin d'anticiper les débordements (objectif < 2 %) | SHOULD | 5 SP

US-013 : En tant que Gestionnaire, je veux accéder à un dashboard analytics (volumes collectés, taux de remplissage, distances, CO2 évité) afin de piloter le service et mesurer les gains | SHOULD | 8 SP

US-014 : En tant qu'Utilisateur, je veux recevoir des notifications push et e-mail (alertes, affectation de tournée, défis) afin d'être informé sans surveiller l'écran | SHOULD | 5 SP

US-015 : En tant qu'Agent de collecte, je veux signaler une anomalie terrain (conteneur endommagé, accès bloqué) avec commentaire et photo afin d'alerter le gestionnaire | SHOULD | 3 SP

US-016 : En tant que Système/API, je veux exposer une API REST documentée (OpenAPI/Swagger) avec rate limiting (100 req/min/IP) afin de garantir l'intégration et la robustesse de la plateforme | SHOULD | 5 SP

US-017 : En tant que Gestionnaire, je veux générer un rapport mensuel PDF (KPIs, graphiques : collectes, distances, débordements) afin d'alimenter le reporting institutionnel | COULD | 5 SP

US-018 : En tant qu'Administrateur, je veux configurer les zones, seuils, capteurs et canaux de notification afin d'adapter la plateforme à l'évolution du terrain | COULD | 3 SP

III. PLANIFICATION PAR SPRINT

Kick-off (S1) & Sprint 0 (S2-3) : Cadrage & Socle Technique

- User Stories : US-002, US-008, US-016 — Total 18 SP
- Objectif : Constituer l'équipe, dresser le backlog initial, cadrer l'architecture applicative et poser le socle technique (repos GitFlow, services Dockerisés, CI/CD GitHub Actions, environnements dev/staging) ainsi que l'ingestion MQTT initiale
- Livrables (DEV) : backlog priorisé MoSCoW + squelette API REST (FastAPI/Node.js) + conteneurs Docker + pipeline GitHub Actions (build/test/lint) + broker MQTT opérationnel + authentification JWT/MFA + RBAC initial

Sprints 1-2 (S4-7) : Fondations & Modules Core

- User Stories : US-003, US-005, US-006, US-009 — Total 29 SP
- Objectif : Livrer le cœur fonctionnel — supervision temps réel des conteneurs et premier circuit agent
- Livrables (DEV) : carte temps réel des conteneurs (React/TypeScript + Leaflet, WebSocket) + persistance des mesures IoT + RBAC 4 rôles complet + application mobile agent (réception de tournée, scan QR, validation de collecte) + premiers tests automatisés

Sprints 3-4 (S8-11) : Fonctionnalités Avancées

- User Stories : US-001, US-004, US-007, US-010, US-011, US-015 — Total 37 SP
- Objectif : Déployer l'optimisation des tournées et l'engagement citoyen, fiabiliser le terrain
- Livrables (DEV) : moteur d'optimisation TSP + 2-opt (feuilles de route) + application mobile citoyen (signalement + photo) + mode hors-ligne agent avec synchronisation + gamification complète (points, badges 30+, défis, leaderboard) + signalement d'anomalie agent

Sprint 5 (S12-14) : Intégration, Tests & Polish

- User Stories : US-012, US-013, US-014, US-016 — Total 23 SP
- Objectif : Industrialiser, intégrer les briques et atteindre les exigences non fonctionnelles
- Livrables (DEV) : dashboard analytics (volumes, taux de remplissage, distances, CO2 évité) + moteur d'alertes sur seuils + notifications push/e-mail + API REST documentée (Swagger) avec rate limiting 100 req/min/IP + montée de la couverture de tests > 60 % + tests de charge (500 users simultanés, API < 500 ms p95)

Soutenance (S15-16) : Recette, Documentation & Présentation

- User Stories : US-017, US-018 — Total 8 SP
- Objectif : Finaliser, documenter et préparer la présentation finale au jury
- Livrables (DEV) : rapports mensuels PDF + backoffice de configuration (zones, seuils, capteurs, canaux) + DCT (30-35 pages) + README + documentation API + code GitHub + application déployée + démonstration de soutenance

IV. OUTILS DE GESTION

- Outil principal : Jira ou GitHub Projects
- Communication : Slack ou Discord
- Versioning : GitHub avec GitFlow
- CI/CD : GitHub Actions ou GitLab CI
- Documentation : Confluence ou Notion

V. INDICATEURS CLÉS DE PERFORMANCE (KPI)

A. KPIs Techniques (filière Développement)

1. Couverture de tests : $(\text{Lignes testées} / \text{Total}) \times 100$ | Cible $> 60 \%$ | Par sprint
2. Temps de réponse API : Latence des endpoints critiques | Cible < 500 ms (p95) | Temps réel
3. Disponibilité système : Uptime de la plateforme | Cible $\geq 99,5 \%$ | Journalier
4. Performance requêtes SQL : Temps d'exécution | Cible < 200 ms pour 90 % des requêtes | Temps réel
5. Vitesse sprint : Story Points complétés par sprint | Cible 25 ± 3 SP | Par sprint
6. Bugs critiques : Nombre de bugs P1 ouverts | Cible 0 | Journalier
7. Cache hit ratio : $(\text{Requêtes servies par le cache} / \text{Total}) \times 100$ | Cible $> 80 \%$ | Temps réel
8. Temps de chargement page : Affichage initial de l'interface | Cible < 3 s | Par release
9. Charge supportée : Utilisateurs simultanés sans dégradation | Cible ≥ 500 | Par campagne de charge
10. Lead time for changes : Délai commit $\rightarrow$ production | Cible < 2 jours | Par release
11. Fréquence de déploiement : Nombre de déploiements en production | Cible $\geq 1/\text{semaine}$ | Hebdo
12. Taux d'échec des changements : $(\text{Déploiements en échec} / \text{Total}) \times 100$ | Cible $< 15 \%$ | Par release
13. MTTR (rétablissement) : Délai moyen de restauration du service après incident | Cible < 1 h | Par incident

B. KPIs Métier

14. Réduction des distances de collecte : $(\text{KM économisés} / \text{KM avant}) \times 100$ | Cible $\geq 20 \%$ | Mensuel
15. Taux de débordement : $(\text{Conteneurs débordés} / \text{Total}) \times 100$ | Cible $< 2 \%$ | Hebdo
16. Citoyens actifs : Nb d'utilisateurs actifs sur la plateforme | Cible 15 000 | Mensuel
17. Réduction des coûts opérationnels et CO2 : Coûts -15% / émissions -18% | Cible atteinte | Mensuel

VI. GESTION DES RISQUES

Risques Communs (toutes filières) :

- Retard développement fonctionnalité critique | Prob: Moyenne | Impact: Élevé
- Départ membre équipe | Prob: Faible | Impact: Moyen
- Dépassement budget | Prob: Moyenne | Impact: Élevé

Risques Spécifiques (filière Développement) :

- Dette technique accumulée | Mitigation : code reviews systématiques + refactoring planifié
- Volumétrie IoT MQTT mal absorbée (~500k msg/j) | Mitigation : tests de charge dès les Sprints 1-2 + cache + file de messages
- Bugs critiques en production | Mitigation : tests automatisés (>60 %) + environnement de staging + CI/CD

VII. RESSOURCES ET BUDGET

A. Équipe Projet

- Product Owner : 1 personne — 25 % temps
- Scrum Master : 1 personne — 25 % temps
- Développeurs full-stack : 3 personnes — 100 % temps
- Architecte logiciel : 1 personne — 50 % temps

B. Budget Prévisionnel (16 semaines)

- Ressources Humaines : 116 k€
- Infrastructure Cloud : 3 k€
- Outils & Licences : 2 k€
- Formation : 3 k€
- Imprévus (10 %) : 12 k€

TOTAL : 136 k€

VIII. DIAGRAMME DE GANTT

Planning sur 4 mois (16 semaines) suivant les jalons du cahier des charges (jalons en gras) :

- S1 — Kick-off : équipe constituée, backlog initial priorisé | Jalon : lancement projet
- S2-3 — Sprint 0 : architecture applicative, environnements, CI/CD, ingestion MQTT initiale, auth JWT/MFA | Jalon : socle technique livré
- S4-7 — Sprints 1-2 : carte temps réel, RBAC, persistance IoT, app mobile agent (tourn  e/scan QR) | Jalon : modules core
- S8-11 — Sprints 3-4 : optimisation TSP/2-opt, app mobile citoyen (signalement+photo), mode offline, gamification | Jalon : fonctionnalit  s avanc  es
- S12-14 — Sprint 5 : dashboard analytics, alertes, notifications, API document  e, tests de charge & couverture > 60 % | Jalon : int  gration, tests, polish
- S15-16 — Soutenance : rapports, backoffice config, DCT, d  ploiement, d  monstration | Jalon : pr  sentation finale

IX. SYNTHÈSE & AMÉLIORATION

A. Récapitulatif

- Méthodologie : Scrum / sprints de 2 semaines sur 4 mois (jalons CDC : Kick-off, Sprint 0, Sprints 1-5, Soutenance)
- Backlog : 18 user stories priorisées (MoSCoW)
- KPIs : 17 indicateurs (13 techniques + 4 métier)
- Budget : 136 k€ sur 4 mois (16 semaines)

B. Axes d'Amélioration (3-5 axes)

1. Automatisation des tests end-to-end : intégrer une suite E2E (Playwright) dans la CI afin de fiabiliser les parcours critiques (signalement citoyen, tournée agent, optimisation) à chaque déploiement.
2. Évolution de l'algorithme d'optimisation : enrichir le couple TSP/2-opt par une métaheuristique (recuit simulé ou algorithme génétique) afin de pousser le gain kilométrique au-delà des 20 % visés.
3. Observabilité : déployer un stack de monitoring (Prometheus/Grafana, traces distribuées) afin de réduire le MTTR et d'anticiper les dérives de performance de l'ingestion MQTT (~500k msg/j).

PROJET ECOTRACK

AUDIT INITIAL

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822)

I. OBJECTIFS DE L'AUDIT INITIAL

✓ Critères RNCP Ce1.1.1 / A1.1 : Évaluation initiale de l'existant

Avant d'engager les ressources humaines et financières du projet ECOTRACK, un audit initial est indispensable pour objectiver l'état des lieux et fonder les décisions sur des faits plutôt que sur des intuitions. La métropole gère aujourd'hui la collecte de 2 000 conteneurs pour 500 000 habitants selon des tournées calendaires, sans aucune visibilité sur le niveau de remplissage réel. Cet audit doit cartographier les forces et faiblesses du contexte, identifier les risques susceptibles de compromettre un projet ambitieux mené en 16 semaines (8 sprints), et sécuriser ainsi la trajectoire de mise en œuvre dès le premier sprint. Les enjeux sont majeurs. Sur le plan technique, ECOTRACK combine des briques complexes : ingestion temps réel de environ 500 000 messages par jour (de mesures par an via MQTT, stockage de séries temporelles et données géospatiales, et optimisation de tournées par algorithme TSP. Sur le plan organisationnel, la réussite dépend de l'adoption par les chauffeurs et de l'intégration au SI existant. L'audit a donc pour finalité d'anticiper les obstacles, de prioriser les actions à fort impact immédiat (quick wins) et de formuler des recommandations stratégiques garantissant que le projet démarre sur des bases saines, maîtrisées et conformes aux exigences réglementaires.

A. Contexte de l'Audit

- Absence de visibilité : Conteneurs non connectés, tournées fixes
- Inefficacité opérationnelle : Débordements fréquents, coûts élevés
- Opportunité IoT : 2000 capteurs à déployer, optimisation algorithmique
- Besoin stratégique : Modernisation gestion déchets urbains

B. Périmètre de l'Audit

- Analyse SWOT (Forces, Faiblesses, Opportunités, Menaces)
- Identification risques techniques et organisationnels (10-12 risques)
- Quick Wins : Actions rapides à fort impact (5-7 actions)
- Recommandations stratégiques pour maximiser chances succès

II. ANALYSE SWOT

✓ Critères RNCP Ce1.1.2 / A1.1 : Analyse stratégique

A. FORCES (Strengths)

Forces Communes (Toutes Filières) :

1. Équipe compétente et motivée (7 personnes, 5.5 ETP)
2. Méthodologie Agile maîtrisée (Scrum, 8 sprints × 2 semaines)
3. Budget alloué cohérent avec ambitions projet
4. Sponsor projet engagé (Directeur Gestion Déchets)
5. Technologies matures et éprouvées
6. Cahier des charges détaillé et réaliste

Forces Spécifiques (Filière Développement) :

DEV : Stack moderne et typée (React 18 + TypeScript, FastAPI/Node.js), architecture microservices scalable, CI/CD automatisé via GitHub Actions dès le Sprint 1, patterns de conception maîtrisés, containerisation Docker pour des environnements reproductibles.
DEV : Couverture de tests automatisés visée dès le Sprint 1 (unitaires, intégration, E2E) et qualité de code outillée (linting, revues, analyse statique), garantissant un produit maintenable et fiable.

B. FAIBLESSES (Weaknesses)

Faiblesses Communes :

1. Délai serré (16 semaines) pour complexité projet
2. Dépendance fournisseurs cloud (AWS/Azure/GCP)
3. Résistance potentielle utilisateurs finaux (chauffeurs)
4. Manque expérience IoT dans l'équipe initiale
5. Intégration SI existants potentiellement complexe

Faiblesses Spécifiques (Filière Développement) :

DEV : Complexité de l'optimisation TSP sur 2 000 points, dette technique potentielle si les sprints sont pressés, tests E2E chronophages à maintenir, courbe d'apprentissage de l'ingestion MQTT haute fréquence, gestion fine du stockage TimescaleDB/PostGIS.

C. OPPORTUNITÉS (Opportunities)

1. Extension géographique : Déploiement autres villes (scalabilité nationale/internationale)
2. Intégration IA prédictive : ML pour prévision remplissage, maintenance prédictive
3. Partenariats collectivités : Marchés publics, mutualisation inter-villes
4. Réduction empreinte carbone : Image RSE, labels environnementaux

5. Innovation services citoyens : App mobile tri déchets, gamification
6. Économies d'échelle : Mutualisation infrastructure cloud
7. Open Data : APIs publiques pour chercheurs, startups, citoyens

D. MENACES (Threats)

1. Concurrence startups GreenTech (Rubicon, Sensoneo, Compology)
2. Évolution réglementations (RGPD renforcé, directives ANSSI)
3. Cyberattaques ciblées (ransomware, DDoS, APT)
4. Pannes capteurs IoT (fiabilité matériel, vandalisme)
5. Budget insuffisant (dépassement coûts cloud, inflation)
6. Turnover équipe (départ compétences clés, marché tension)
7. Adoption faible utilisateurs (résistance changement, UX inadaptée)

III. IDENTIFICATION DES RISQUES

✓ Critères RNCP Ce1.5.2 / A1.5 : Gestion des risques

A. Risques Communs (Toutes Filières)

RISQUE 1 : Retard développement fonctionnalité critique

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : MVP simplifié (algo glouton avant génétique), priorisation MoSCoW stricte, buffer 10% estimations, daily stand-ups identification blocages précoce.

RISQUE 2 : Départ membre équipe clé

Prob : Faible | Impact : Élevé | Criticité : MOYENNE

Mitigation : Documentation continue (ADR, README), pair programming systématique, knowledge sharing hebdo, rétention talents (reconnaissance, bonnes conditions travail).

RISQUE 3 : Dépassement budget

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : Suivi hebdomadaire budget détaillé, alertes automatiques à 80%, réserve contingence 10%, optimisation ressources cloud (shutdown env non-prod nuit/weekend).

RISQUE 4 : Résistance utilisateurs finaux

Prob : Moyenne | Impact : Moyen | Criticité : MOYENNE

Mitigation : Beta testing avec 10 chauffeurs pilotes dès Sprint 4, formation progressive personnalisée, feedback loops réguliers, gamification avec récompenses.

B. Risques Spécifiques (Filière Développement)

Risques Applicatifs (Développement) :

RISQUE 5 : Dette technique accumulée

Prob : Moyenne | Impact : Moyen | Criticité : MOYENNE

Mitigation : Code reviews obligatoires (2 reviewers), refactoring sprints dédiés (15% vélocité), SonarQube intégré CI, pair programming features complexes.

RISQUE 6 : Scalabilité insuffisante (2000 capteurs)

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : Load testing dès Sprint 3 (10K req/min), architecture microservices découplée, file messages RabbitMQ/Kafka, cache Redis agressif, auto-scaling Kubernetes.

RISQUE 7 : Bugs critiques en production

Prob : Faible | Impact : Élevé | Criticité : MOYENNE

Mitigation : Tests auto (unit 80%, intégration, E2E), staging identique prod, rollback auto via CI/CD, monitoring APM (Sentry), feature flags progressive rollout.

RISQUE 8 : Montée en charge de l'ingestion IoT (~500 000 messages/jour)

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : broker MQTT Mosquitto dimensionné, mise en file et traitement asynchrone, écritures en lot dans TimescaleDB (hypertables, compression), tests de charge dès le Sprint 3 (objectif 2 000 msg/min soutenus), supervision du backlog de messages avec alertes.

RISQUE 9 : Complexité et performance de l'algorithme TSP

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : heuristique gloutonne pour le MVP avant algorithme génétique, calcul asynchrone hors du chemin critique, cache Redis des matrices de distances, contraintes métier paramétrables (capacité véhicules, zones), benchmarks de qualité et de temps de calcul à chaque sprint.

RISQUE 10 : Non-conformité RGPD des données de géolocalisation

Prob : Moyenne | Impact : Élevé | Criticité : ÉLEVÉE

Mitigation : privacy by design, minimisation et pseudonymisation des données de tournées et chauffeurs, chiffrement at-rest (AES-256) et in-transit (TLS 1.3), registre de traitement, DPIA validée et durées de rétention définies avant mise en production.

RISQUE 11 : Intégration complexe au SI existant de la métropole

Prob : Moyenne | Impact : Moyen | Criticité : MOYENNE

Mitigation : cadrage des interfaces dès le Sprint 1, API contractualisées (OpenAPI/Swagger), connecteurs découplés et idempotents, environnement de staging miroir, tests d'intégration automatisés et plan de bascule progressif validé avec la DSI.

RISQUE 12 : Dépendance forte au fournisseur cloud

Prob : Faible | Impact : Moyen | Criticité : MOYENNE

Mitigation : architecture portable et containerisée (Docker), recours privilégié à des briques open source (PostgreSQL, Redis, Mosquitto), infrastructure as code pour reproductibilité, abstraction des services managés et clauses de réversibilité contractuelles.

IV. QUICK WINS (Actions Rapides à Fort Impact)



Critères RNCP Ce1.4.1 / A1.4 : Priorisation actions efficaces

A. Quick Wins Communs (Toutes Filières)

QW1 : Setup CI/CD avec GitHub Actions

Effort : 1 jour | Impact : Élevé

Bénéfice : Tests automatisés + déploiement continu → gain temps 30%, qualité code↑, détection bugs précoce

QW2 : Documentation projet complète (README + CONTRIBUTING)

Effort : 4 heures | Impact : Moyen

Bénéfice : Onboarding rapide nouveaux devs (<1h), alignement équipe sur standards, réduction questions répétitives

QW3 : Rituels Scrum quotidiens (Daily Stand-ups 15 min)

Effort : 0 (rituels standards) | Impact : Moyen

Bénéfice : Communication équipe↑, blocages identifiés tôt (<24h), vélocité sprint↑ 10-15%

B. Quick Wins Spécifiques (Filière Développement)

Quick Wins Développement :

QW4 : Linting automatique + pre-commit hooks (ESLint, Prettier)

Effort : 2h | Impact : Moyen | Bénéfice : Qualité code↑, bugs↓ 20%, style uniforme, reviews simplifiées

QW5 : Cache Redis pour requêtes fréquentes

Effort : 1 jour | Impact : Élevé | Bénéfice : Temps de réponse API optimisé (cache, requêtes), p95 maintenu sous la cible de 500 ms, scalabilité↑, charge BDD↓

QW6 : Pagination API (limit/offset) sur endpoints liste

Effort : 4h | Impact : Élevé | Bénéfice : Scalabilité instantanée, performances frontend↑, UX améliorée

QW7 : WebSocket push notifications temps réel

Effort : 1 jour | Impact : Moyen | Bénéfice : UX moderne, temps réel natif (alertes débordements instantanées)

V. RECOMMANDATIONS STRATÉGIQUES



Critères RNCP CeI.6.4 / AI.6 : Amélioration continue

Recommandations Priorité 1 (CRITIQUE) :

R1. Approche MVP (Minimum Viable Product)

- Lancer avec fonctionnalités essentielles (cartographie + alertes + tournées basiques)
- Itérer rapidement basé feedback utilisateurs réels (beta Sprint 4)
- Éviter perfectionnisme retardant production (done > perfect)
- Mesurer adoption : NPS, usage stats, taux complétion tâches

R2. Tests et Validation Continue

- Tests unitaires $\geq 80\%$ dès Sprint 1 (TDD/BDD obligatoire)
- Tests intégration + E2E parcours critiques (Cypress/Playwright)
- CI/CD validation automatique avant merge (GitHub Actions)
- Load testing Sprints 3, 6, 8 : 500 utilisateurs simultanés, API < 500 ms (p95)

R3. Sécurité Native (Security by Design)

- Privacy by design dès conception (RGPD native, data minimization)
- Chiffrement obligatoire (TLS 1.3 in-transit + AES-256 at-rest)
- Audit sécurité externe avant prod (pentest, code review sécu)
- DPIA (Data Protection Impact Assessment) validée CNIL

Recommandations Priorité 2 (IMPORTANT) :

R4. Implication Continue Utilisateurs Finaux

- Beta testing dès Sprint 4 avec 10 chauffeurs pilotes volontaires
- Feedback loops hebdomadaires (questionnaires + interviews 30 min)
- Formation progressive pré-déploiement (2 sessions 2h pratiques)
- Champions utilisateurs internes (ambassadeurs, early adopters)

R5. Documentation Technique Continue

- Architecture Decision Records (ADR) pour choix techniques majeurs
- README.md actualisé + doc API (OpenAPI/Swagger auto-générée)
- Runbooks incidents courants (playbooks SOC, procédures debug)
- Diagrammes architecture à jour (draw.io, C4 model)

R6. Monitoring et Observabilité 360°

- Métriques : Prometheus + Grafana (infra + app + métier)
- Logs centralisés : ELK Stack + rétention 90 jours
- Traces distribuées : APM (Sentry / DataDog) pour debugging
- Alertes intelligentes : PagerDuty + escalade automatique

R7. Plan Communication Proactif

- Rapport hebdomadaire sponsors (KPIs vitesse, risques, décisions)
- Démonstrations sprint toutes les 2 semaines (Sprint Review publique)
- Transparence totale difficultés (no surprises, honnêteté)
- Célébration succès équipe (reconnaissance publique, team building)

Recommandations Priorité 3 (SOUHAITABLE) :

R8. Architecture Scalable Dès Début

- Microservices + containerisation (évolutions futures facilitées)
- Load testing précoce Sprint 3 (anticiper goulots performance)
- Design for failure (circuit breakers, retries exponentiels, timeouts)

R9. Formation Continue Équipe

- Veille techno hebdomadaire (1h/dev, partage découvertes)
- Tech talks mensuels internes (knowledge sharing équipe)
- Certifications cloud encouragées (AWS SA, Azure Admin)

R10. Préparation Scalabilité Future

- Multi-tenancy anticipé (architecture isolée par client/ville)
- Internationalisation (i18n) dès Sprint 1 (fr, en prévus)
- API versioning (v1, v2...) pour évolutions sans rupture

VI. CONCLUSION DE L'AUDIT

L'audit initial confirme que le projet ECOTRACK repose sur des fondations solides. L'équipe de 7 personnes (5,5 ETP) maîtrise la méthodologie Scrum et la stack retenue (React 18 + TypeScript, FastAPI/Node.js, PostgreSQL/PostGIS/TimescaleDB, Redis, MQTT, Docker). Les technologies sont matures, open source et parfaitement adaptées aux contraintes géospatiales, temps réel et séries temporelles du projet. Le cahier des charges est détaillé et le sponsor pleinement engagé.

L'analyse SWOT met en évidence des opportunités significatives (extension à d'autres villes, IA prédictive, valorisation Open Data) tout en pointant des faiblesses à surveiller, principalement la complexité de l'algorithme TSP, le délai serré de 16 semaines et l'intégration au SI existant. Ces points de vigilance sont connus et adressables.

Les 12 risques identifiés — du retard de développement à la conformité RGPD des données de géolocalisation, en passant par la montée en charge de l'ingestion IoT et la dette technique — sont tous assortis de plans de mitigation concrets et actionnables. Les quick wins (CI/CD, cache Redis, pagination, linting, WebSocket) délivrent un impact élevé dès les premiers jours, et les recommandations stratégiques priorisées (approche MVP, tests continus, sécurité native) structurent une trajectoire de réussite.

En synthèse, la faisabilité du projet est confirmée. Sous réserve du respect d'une approche MVP, d'un suivi hebdomadaire des risques et du budget, et de l'intégration de la sécurité et de la conformité dès la conception, ECOTRACK peut démarrer son premier sprint dans des conditions maîtrisées.

Points Clés à Retenir :

- Forces significatives : Équipe compétente, méthodologie éprouvée, stack mature
- Risques identifiés et maîtrisables : 12 risques avec plans de mitigation définis
- Quick wins actionnables : 7 actions rapides à impact élevé dès le Sprint 1
- Recommandations stratégiques : 10 recommandations priorisées pour succès
- Opportunités croissance : Extension géographique, IA prédictive, partenariats

FAISABILITÉ PROJET : FAVORABLE

DÉCISION : GO POUR DÉMARRAGE SPRINT 1

————— **FIN DE L'AUDIT INITIAL** —————